Problem Set 6: Partial Differential Equations and Fourier Methods

Readings: Newman Ch. 7, Ch. 9

Due: Wednesday, November 11th 11:59 PM

Instructions: Work through each problem, following the instructions for the requirements of the code and their outputs. Some starting code is provided for each problem on the course website. Problem sets should be submitted in one batch, with each problem uploaded as it's own python script (`.py` file) on the HuskyCT assignment page with the filename: `ps[PROBLEM SET NUMBER]_[QUESTION NUMBER]_[LASTNAME].py`.

Parameters:

(1) Submitted solutions are restricted to imports of external functions from standard built-in Python libraries (i.e. `math, time, os, sys`, etc.). In addition, imports from `matplotlib.pyplot` and `numpy` are permitted for basic array operations, unless otherwise stated.

(2) All plots should have both axes clearly labeled and include a title. Plots should include either legends or annotations for differentiating plot elements, when applicable.

(3) You can use the appropriate functions from the `numpy.fft` library to perform the fourier transforms on this problem set.

---

1. THERMAL DIFFUSION IN THE EARTH'S CRUST

A classic example of a diffusion problem with a time-varying boundary condition is the diffusion of surface heat into the crust of the Earth. Suppose the mean daily temperature at a particular point on the surface varies as:

$$T_0(t) = A + B\sin\left[\frac{2\pi t}{\tau} - \pi/2\right],$$

where $\tau = 365$ days, $A = 10^\circ$C and $B = 12^\circ$C. At a depth of 20 m below the surface almost all seasonal temperature variation is ironed out and the temperature is, to a good approximation, a constant 11°C (which is higher than the mean surface temperature of 10°C — temperature increases with depth, due to heating from the hot core of the planet). The thermal diffusivity of the Earth's crust varies somewhat from place to place, but for our purposes we will treat it as constant with value $D = 0.1\,\mathrm{m}^2\,\mathrm{day}^{-1}$.

Write a program to calculate the temperature profile of the Earth's crust as a function of depth and time. Your program should:

a. Include a function to calculate the temperature profile from the surface down to 20 m, evolving to a given time t, by solving the heat diffusion equation.

b. Assume that initially, the temperature is equal everywhere to the mean surface temperature, impose the appropriate boundary conditions, and run the calculation for 10 years with an appropriate grid and time spacing.

c. Plot four temperature profiles taken at 3-month intervals for the last year of the calculation on a single plot to illustrate how temperature varies as a function of depth and time. Label each line with it's corresponding season.

d. Save this figure as `crustheat_[LASTNAME].png`.

2. THE SCHRÖDINGER EQUATION AND THE SPECTRAL METHOD

This exercise uses the spectral method to solve the time-dependent Schödinger equation

$$-\frac{\hbar^2}{2M}\frac{\partial^2\psi}{\partial x^2} = \hbar\frac{\partial\psi}{\partial t}$$

for a single particle in one dimension in a box of length L with impenetrable walls. The wavefunction in such a box necessarily goes to zero on the walls and hence one possible (unnormalized) solution of the equation is

$$\psi_k(x,t) = \sin\left(\frac{\pi k x}{L}\right) e^{iEt/\hbar},$$

where the energy E can be found by substituting into the Schrödinger equation, giving

$$E = \frac{\pi^2\hbar^2k^2}{2mL^2}.$$

As with the vibrating string in Newman 9.3.4, we can write a full solution as a linear combination of such individual solutions, which on the grid points $x_n = nL/N$ takes the value

$$\psi(x_n,t) = \frac{1}{N}\sum_{k=1}^{N-1} b_k \sin\left(\frac{\pi k n}{N}\right) \exp\left(i\frac{\pi^2\hbar k^2}{2mL^2}t\right),$$

where the b_k are some set of (possibly complex) coefficients that specify the exact shape of the wavefunction and the leading factor of $1/N$ is optional but convenient.

Consider an electron (mass $m_e = 9.109\times10^{-31}$ kg) in a box of length $L = 10^{-8}$ m. At time $t = 0$, the wavefunction of the electron has the form

$$\psi(x,0) = \exp\left[-\frac{(x-x_0)^2}{2\sigma^2}\right] e^{i\kappa x},$$

where

$$x_0 = \frac{L}{2}, \qquad \sigma = 1\times10^{-10}\ \text{m}, \qquad \kappa = 5\times10^{10}\ \text{m}^{-1}.$$

Write a program to use the spectral method to solve for the time evolution of the wavefunction. Your program should:

a. Calculate the values of the coefficients in terms of their real and imaginary parts: $b_k = \alpha_k + i\eta_k$ at $N = 1000$ grid points for all $k = 1\ldots N-1$. To perform the discrete sine transforms, you can use the fast transform function `dst` from the provided package `dcst.py`.

b. Include a function to calculate the real part of the wavefunction:

$$\Re\psi(x_n,t) = \frac{1}{N}\sum_{k=1}^{N-1}\left[\alpha_k\cos\left(\frac{\pi^2\hbar k^2}{2ML^2}t\right) - \eta_k\sin\left(\frac{\pi^2\hbar k^2}{2ML^2}t\right)\right]\sin\left(\frac{\pi k n}{N}\right)$$

at some arbitrary time t using the inverse discrete sine transform function `idst`, from `dcst.py`.

c. Calculate and plot the time evolution of the real part of the wavefunction starting at $t = 10^{-16}$ s. You can choose how to demonstrate the time-evolution, but your plot should have appropriate supporting annotation/labels and print a caption to communicate a physical interpretation of the behavior.

Hint: You might want to try a time interval between graphs in multiples of 10^{-16} s.

d. Save the figure as `spectralwave_[LASTNAME].png`.

3. DETECTING PERIODICITY

Let's re-analyze the sunspot data (from PS #2) with a Fourier analysis.
Write a program to find the period of the sunspot cycle using a Fourier transform. Your program should:

a. Read in the data from the `sunspots.txt` file, which contains the observed number of sunspots on the Sun for each month since January 1749.

b. Calculate the Fourier transform of the sunspot data and plot the magnitude squared $|c_k|^2$ of the Fourier coefficients as a function of k—also called the *power spectrum* of the sunspot signal.

c. Save this plot as `sunspotFT_[LASTNAME].png`.

d. Find the approximate value of k_{peak}, the noticeable peak in the power spectrum at a nonzero value of k which tells us that there is one frequency in the Fourier series that has a higher amplitude than the others around it. Calculate and print the corresponding period for k_{peak} in years .

e. While there is periodic behavior, over longer timescales there are secular trends (Like if you added a sine wave to a slowly varying monotonic function). We can isolate the periodic behavior by de-trending the data. Use your `running_average` function from PS2 with an informed window size to help you calculate the longer timescale variation trend and subtract it off from the sunspot data. Plot the detrended sunspot data on a subplot.

f. Demonstrate the periodicity of the detrended data by phase-folding it: on a second subplot, plot the detrended sunspot number against the phase: $\phi = \frac{(t-t_0)}{P} \% N$ where P is the period you calculated, $\%$ is the modulo operator, and N is the number of periods to fold across.

g. Overplot a sine curve of the form $A\sin(2\pi\phi) + C$ on the phase-folded data plot, where the constants A and C should reflect a typical amplitude variation and median sunspot number, respectively. Note that the data have significant period to period maximum amplitude variation, so there will be some spread, but there should be a clear periodicity visible.

h. Save this figure as `sunspotmodel_[LASTNAME].png`.

4. IMAGE DECONVOLUTION

When a photo is blurry each point on the photo gets smeared out according to some "smearing distribution," called a *point spread function*. For simplicity let's assume we're working with a black and white photograph, so that the unblurred image can be represented by a single function $a(x,y)$ which tells you the brightness at each point (x,y). And let us denote the point spread function by $f(x,y)$ in which a point source will end up appearing as $f(x,y)$ instead.

In general the brightness $b(x,y)$ of the blurred image at point (x,y) is given by

$$b(x,y) = \int_0^K \int_0^L a(x',y') f(x-x', y-y')\, dx'\, dy',$$

where $K \times L$ is the dimension of the picture. This equation is called the *convolution* of the image with the point spread function. If $f(x,y)$ is a broad function then the resulting picture will be badly blurred. If it is a narrow peak then the picture will be relatively sharp.

We can represent the brightness as a Fourier series, and show that apart from the factors of KL, the Fourier transform of the blurred photo is the product of the Fourier transforms of the unblurred photo and the point spread function. For a picture of dimensions $K \times L$ we find that the two-dimensional Fourier transforms are related by

$$\tilde{b}_{kl} = KL\tilde{a}_{kl}\tilde{f}_{kl}\,,$$

Then to deblur a picture, we take the blurred picture and Fourier transform it. We also take the point spread function and Fourier transform it to get $\tilde{f}_{kl}$. Then we divide one by the other:

$$\frac{\tilde{b}_{kl}}{KL\tilde{f}_{kl}} = \tilde{a}_{kl}$$

which gives us the Fourier transform of the *unblurred* picture. Then, finally, we do an inverse Fourier transform on $\tilde{a}_{kl}$ to get back the unblurred picture. This process of recovering the unblurred picture from the blurred one, of reversing the convolution process, is called *deconvolution*.

For many cameras, it's a reasonable approximation to assume the point spread function is a Gaussian:

$$f(x,y) = \exp\left(-\frac{x^2+y^2}{2\sigma^2}\right),$$

where σ is the width of the Gaussian.



Figure 1: The point spread function should be periodic on the interval in both dimensions. That is, if you visualize the PSF used for de-convolution, it should look something like this.

Write a program to deblur an image using the outlined procedure above and the numpy methods. Your program should:

a. Read in the file `blur.txt` which contains a grid of values representing brightness on a black-and-white photo—a badly out-of-focus one that has been deliberately blurred using a Gaussian point spread function of width $\sigma = 25$.

b. Include a function that constructs an array of the same size as the photo for the PSF $f(x,y)$ with $\sigma = 25$.

c. Fourier transform both the blurred image and the PSF and calculates the Fourier transform of the unblurred image.

d. Perform an inverse Fourier transform to deconvolve the image.

e. Plot the blurred and unblurred image side by side using a density plot. Save this image as `deblur_[LASTNAME].png`.

Hint: If you make a plot, the picture has the sky, which is bright, at the top and the ground, which is dark, at the bottom.

Hint: One thing you'll need to deal with is what happens when the Fourier transform of the point spread function is zero, or close to zero. A workable compromise is that if a value in the Fourier transform of the point spread function is smaller than a certain amount ϵ you don't divide by it—just leave that coefficient alone. The value of ϵ is not very critical but a reasonable value seems to be 10^{-3}.